

Trabalho Prático II

Enzo Borges Segala
Rafael Silveira Bandeira
Jonathan Gabriel Nunes Mendes
Matheus Machado Cezar
Marcos Henrique Volpato De Moraes
Miguel Predebon Abichequer
Nicolas Rosenthal Dal Corso

1. Compressão baseada no modo básico do JPEG

Nesta etapa do trabalho foi implementada uma versão simplificada do método JPEG *baseline*, mantendo as operações essenciais do algoritmo, porém sem utilizar a etapa de codificação entrópica (Run-Length Encoding e codificação de Huffman). O objetivo é analisar o comportamento da compressão apenas com as etapas de transformação, quantização e reconstrução.

1.1 Metodologia

A compressão foi aplicada a duas imagens em tons de cinza selecionadas previamente. O método adotado segue a lógica do JPEG básico, composta pelas seguintes etapas:

a) Preparação da imagem

A imagem original é convertida para valores do tipo *double* e, caso suas dimensões não sejam múltiplas de 8, aplica-se *padding* replicando os valores da borda. Isso garante que a imagem possa ser dividida integralmente em blocos de 8×8 pixels, tamanho padrão do JPEG.

b) Transformada Discreta do Cosseno (DCT)

Após a divisão em blocos, é aplicada a DCT bidimensional em cada bloco. Essa transformada converte a informação espacial da imagem em componentes de frequência, concentrando grande parte da energia nos primeiros coeficientes.

c) Quantização

Cada bloco transformado é quantizado utilizando uma matriz de quantização padrão do JPEG, escalada por um fator **Q**, que controla a intensidade da compressão. Valores maiores de **Q** provocam quantização mais agressiva e, portanto, maior perda de informação e maior taxa de compressão.

d) Reconstrução (Descompressão)

Para reconstruir a imagem comprimida, os coeficientes quantizados são de-quantizados (multiplicação pela matriz), e a Transformada Discreta do Cosseno inversa (IDCT) é aplicada aos blocos, retornando ao domínio espacial.

e) Cálculo das métricas

Dois indicadores foram utilizados para avaliar os resultados:

- **Taxa de compressão:** estimada considerando a proporção entre o número total de coeficientes e o número de coeficientes não nulos após a quantização.
- **PSNR (Peak Signal-to-Noise Ratio):** mede a similaridade entre a imagem original e a reconstruída; valores maiores indicam menor perda de qualidade.

1.2 Código

```
function [rec_img, ratio, psnr_val] = jpeg_basic(img, Q)

    img = double(img);
    [h, w] = size(img);

    % Ajustar dimensões para múltiplos de 8
    H = ceil(h/8) * 8;
    W = ceil(w/8) * 8;

    padded = padarray(img, [H-h W-w], 'replicate', 'post');

    % Matriz de quantização base do JPEG
    Qbase = [16 11 10 16 24 40 51 61;
             12 12 14 19 26 58 60 55;
             14 13 16 24 40 57 69 56;
             14 17 22 29 51 87 80 62;
             18 22 37 56 68 109 103 77;
             24 35 55 64 81 104 113 92;
             49 64 78 87 103 121 120 101;
             72 92 95 98 112 100 103 99];

    Qmatrix = Q * Qbase;

    coeffs = zeros(H,W);

    % --- Compressão: DCT + Quantização ---
    for i = 1:8:H
        for j = 1:8:W
            bloco = padded(i:i+7, j:j+7);
            D = dct2(bloco);
            C = round(D ./ Qmatrix);
            coeffs(i:i+7, j:j+7) = C;
        end
    end

    % Estimar taxa de compressão
    total = numel(coeffs);
    nzeros = nnz(coeffs);
    ratio = total / nzeros;

    % --- Descompressão ---
    rec = zeros(H,W);
    for i = 1:8:H
        for j = 1:8:W
            C = coeffs(i:i+7, j:j+7);
            D = C .* Qmatrix;
            rec(i:i+7, j:j+7) = idct2(D);
        end
    end

    rec_img = rec(1:h, 1:w);

    % --- PSNR ---
    mse = mean((img(:) - rec_img(:)).^2);
    psnr_val = 10 * log10(255^2 / mse);

end
```

```

clear all; close all; clc;

% Carregar imagens em tons de cinza
img1 = imread('borboleta.jpg');
img2 = imread('raposa.jpg');

if size(img1,3) == 3
    img1 = rgb2gray(img1);
end
if size(img2,3) == 3
    img2 = rgb2gray(img2);
end

% Valores de Q para teste
Qvalues = [2 4 8 16];

for k = 1:length(Qvalues)

    Q = Qvalues(k);
    fprintf('\n=== Testando com Q = %d ===\n', Q);

    % Processar imagem 1
    [recl, ratiol, psnr1] = jpeg_basic(img1, Q);
    fprintf('Imagem 1 - Taxa: %.2f | PSNR: %.2f dB\n', ratiol, psnr1);

    figure;
    subplot(1,2,1); imshow(uint8(img1)); title('Original');
    subplot(1,2,2); imshow(uint8(recl)); title(['Reconstruida Q=' num2str(Q)]);

    % Processar imagem 2
    [rec2, ratio2, psnr2] = jpeg_basic(img2, Q);
    fprintf('Imagem 2 - Taxa: %.2f | PSNR: %.2f dB\n', ratio2, psnr2);

    figure;
    subplot(1,2,1); imshow(uint8(img2)); title('Original');
    subplot(1,2,2); imshow(uint8(rec2)); title(['Reconstruida Q=' num2str(Q)]);

end

```

1.3 Resultados Obtidos

A compressão foi aplicada utilizando quatro valores diferentes para o fator de quantização (**Q = 2, 4, 8 e 16**) de modo a observar como o aumento da quantização influencia tanto a taxa de compressão quanto a qualidade final da imagem reconstruída.

1.3.1 Imagens originais



1.3.1 Imagens resultantes

Imagem 1: borboleta.jpg		
Q	Taxa de Compressão	PSNR (dB)
2	11.89	31.95
4	20.82	29.64
8	35.33	27.24
16	52.01	24.42

Q = 2 | PSNR = 31.95 | Taxa = 11.89



Q = 4 | PSNR = 29.64 | Taxa = 20.82



Q = 8 | PSNR = 27.24 | Taxa = 35.33



Q = 16 | PSNR = 24.42 | Taxa = 52.01



Imagem 2: raposa.jpg		
Q	Taxa de Compressão	PSNR (dB)
2	12.55	29.38
4	20.58	27.80
8	34.36	26.03
16	52.65	23.65

Q = 2 | PSNR = 29.38 | Taxa = 12.55



Q = 4 | PSNR = 27.80 | Taxa = 20.58



Q = 8 | PSNR = 26.03 | Taxa = 34.36



Q = 16 | PSNR = 23.65 | Taxa = 52.65



1.4 Análise dos Resultados

Os testes realizados mostram claramente a relação entre o fator de quantização e a qualidade final da imagem:

- **Q baixo (ex.: Q = 2):** Quando a quantização é fraca, a perda de informação é pequena. As duas imagens apresentaram PSNR acima de 29 dB, indicando boa preservação visual. A taxa de compressão, porém, permanece relativamente modesta (cerca de 12%).
- **Q intermediário (Q = 4 e 8):** À medida que o Q aumenta, mais coeficientes da DCT são aproximados para zero, o que reduz o tamanho dos dados. As taxas aumentam para cerca de 20% e depois para mais de 30%, ao custo de uma queda gradual no PSNR. Nessa faixa, os primeiros artefatos típicos do JPEG começam a aparecer, especialmente blocagem em áreas com pouca textura.

- **Q alto (Q = 16):** A compressão torna-se bastante agressiva: as taxas ultrapassam **50%**, ou seja, mais da metade dos coeficientes se tornam nulos. Consequentemente, a qualidade se degrada mais intensamente, com PSNR entre **23 e 24 dB**, valores que normalmente já representam perda de detalhes e blocos visíveis.

Comparando as duas imagens, nota-se que:

- A **Imagem 1** mantém valores de PSNR ligeiramente superiores em todos os níveis de compressão, sugerindo que sua estrutura visual é mais adequada ao modelo JPEG.
- A **Imagem 2** apresenta queda de qualidade um pouco mais acentuada, indicando sensibilidade maior à quantização, possivelmente por conter áreas suaves ou gradientes que são mais afetadas por perdas nas altas frequências.

No geral, os resultados confirmam o comportamento esperado do algoritmo JPEG: **mais compressão → mais perda → menor PSNR**, com artefatos cada vez mais perceptíveis.

2. Compressão baseada na quantização vetorial

Nesta etapa do trabalho, foi implementada uma técnica de compressão baseada em **quantização vetorial (VQ)**. Esse método consiste em dividir a imagem em blocos de dimensões fixas, agrupar blocos similares por meio do algoritmo **K-Means**, e substituir cada bloco da imagem por um índice que referencia um bloco representativo dentro de um *codebook*.

O processo foi aplicado a quatro imagens: duas em tons de cinza (**doca** e **cachorro**) e duas coloridas (**borboleta** e **raposa**). Para cada imagem, foram testados dois tamanhos de blocos: **2×2** e **3×3**. Em todos os casos, o *codebook* é composto por **256 vetores**, conforme solicitado no enunciado.

2.1 Metodologia

O método foi dividido nas seguintes etapas:

a) Formação dos blocos

A imagem é percorrida em blocos **não sobrepostos**, de tamanho 2×2 ou 3×3. Cada bloco é convertido em um vetor coluna. No caso de imagens coloridas, esse processo é feito **por canal (R,G,B)**.

b) Construção do Codebook (K-Means)

Coletamos uma amostra dos blocos da imagem e aplicamos o algoritmo **K-Means** para agrupá-los em 256 grupos. O centro de cada cluster torna-se uma **palavra do codebook**.

c) Compressão

Cada bloco da imagem é substituído por **um índice inteiro entre 1 e 256**, correspondente ao bloco mais próximo do codebook. A imagem comprimida contém: o codebook e a matriz de índices.

d) Reconstrução

A imagem é reconstituída substituindo cada índice pelo bloco correspondente no codebook. Para imagens coloridas, a operação é feita separadamente para os três canais.

e) Métricas

Foram calculados:

- **Taxa de compressão** = bits da imagem original / bits da representação VQ
- **PSNR**, comparando a imagem reconstituída com a original

2.2 Código

```
1  function [rec_img, ratio, psnr_med] = vq_color(img, B)
2
3  -     R = img(:,:,1);
4  -     G = img(:,:,2);
5  -     Bc = img(:,:,3);
6
7  -     [recR, rR, pR] = vq_gray(R, B);
8  -     [recG, rG, pG] = vq_gray(G, B);
9  -     [recB, rB, pB] = vq_gray(Bc, B);
10
11 -     rec_img = cat(3, recR, recG, recB);
12
13 -     ratio = (rR + rG + rB) / 3;
14 -     psnr_med = (pR + pG + pB) / 3;
15
16 - end
17
```

```

1 function [rec_img, ratio, psnr_val] = vq_gray(img, B)
2
3     img = double(img);
4
5     % GARANTIA DE QUE É CINZA (2D)
6     if ndims(img) ~= 2
7         img = rgb2gray(uint8(img));
8         img = double(img);
9     end
10
11     [h, w] = size(img);
12
13     H = ceil(h/B) * B;
14     W = ceil(w/B) * B;
15
16     padded = padarray(img, [H-h, W-w], 'replicate', 'post');
17
18     % EXTRAÍ BLOCOS SEMPRE 2D
19     blocks = im2col(padded, [B B], 'distinct');
20     N = size(blocks, 1);
21
22     max_samples = 10000;
23
24     if N > max_samples
25         idx = randperm(N, max_samples);
26         sample_blocks = blocks(idx, :);
27     else
28         sample_blocks = blocks;
29     end
30
31     K = 256;
32
33     [~, codebook] = kmeans(sample_blocks, K, 'MaxIter', 20, 'Replicates', 1);
34
35     D = pdist2(blocks, codebook);
36     [~, compressed_idx] = min(D, [], 2);
37
38     rec_blocks = codebook(compressed_idx, :);
39     rec_blocks = rec_blocks';
40
41     rec_padded = col2im(rec_blocks, [B B], [H W], 'distinct');
42
43     rec_img = rec_padded(1:h, 1:w);
44
45     bits_original = numel(img) * 8;
46     bits_codebook = numel(codebook) * 8;
47     bits_indices = N * 8;
48
49     bits_total = bits_codebook + bits_indices;
50
51     ratio = bits_original / bits_total;
52
53     mse = mean((img(:) - rec_img(:)).^2);
54     psnr_val = 10 * log10(255^2 / mse);
55
56 end
57

```

```

1 - clear all; close all; clc;
2
3 - gray_imgs = {'doca.jpg', 'cachorro.jpg'};
4 - color_imgs = {'borboleta.jpg', 'raposa.jpg'};
5
6 - B_values = [2 3];
7
8 % IMAGENS EM CINZA
9 - for b = 1:length(B_values)
10 -     B = B_values(b);
11
12 -     figure('Name', ['Cinza B=' num2str(B)], 'NumberTitle', 'off');
13
14 -     for i = 1:length(gray_imgs)
15
16 -         img = imread(gray_imgs{i});
17
18 -         if ndims(img) == 3
19 -             img = rgb2gray(img);
20 -         end
21
22 -         img = double(img);
23 -         img = imresize(img, [256 256]);
24
25 -         [rec, ratio, psnr_val] = vq_gray(img, B);
26
27 -         disp(['Cinza | ' gray_imgs{i} ...
28 -             ' | B=' num2str(B) ...
29 -             ' | TAXA=' num2str(ratio, '%.2f') ...
30 -             ' | PSNR=' num2str(psnr_val, '%.2f')]);
31
32 -         subplot(2,2,i);
33 -         imshow(uint8(img));
34
35 -         subplot(2,2,i+2);
36 -         imshow(uint8(rec));
37 -     end
38 - end
39
40 % IMAGENS COLORIDAS
41 - for b = 1:length(B_values)
42 -     B = B_values(b);
43
44 -     figure('Name', ['Coloridas B=' num2str(B)], 'NumberTitle', 'off');
45
46 -     for i = 1:length(color_imgs)
47
48 -         img = imread(color_imgs{i});
49 -         img = double(img);
50 -         img = imresize(img, [200 200]);
51
52 -         [rec, ratio, psnr_val] = vq_color(img, B);
53
54 -         disp(['Color | ' color_imgs{i} ...
55 -             ' | B=' num2str(B) ...
56 -             ' | TAXA=' num2str(ratio, '%.2f') ...
57 -             ' | PSNR=' num2str(psnr_val, '%.2f')]);
58
59 -         subplot(2,2,i);
60 -         imshow(uint8(img));
61
62 -         subplot(2,2,i+2);
63 -         imshow(uint8(rec));
64 -     end
65 - end

```

2.3 Resultados Obtidos

Os valores de **taxa de compressão** e **PSNR** obtidos estão apresentados nas tabelas a seguir.

Imagens em tons de cinza			
Imagem	Bloco	Taxa	PSNR (dB)
doca	2x2	3.76	36.73
cachorro	2x2	3.76	38.04
doca	3x3	6.76	33.47
cachorro	3x3	6.76	35.23

Original



2x2 | Taxa = 3.76



3x3 | Taxa = 6.76



Original



2x2 | Taxa = 3.76

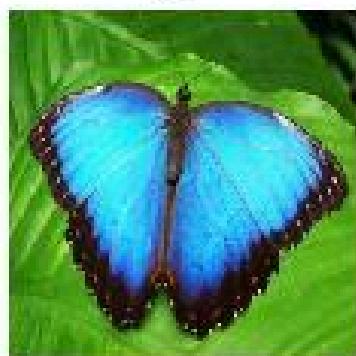


3x3 | Taxa = 6.76

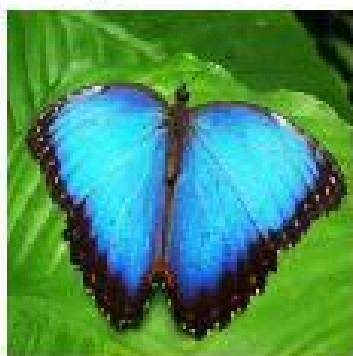


Imagens coloridas			
Imagem	Bloco	Taxa	PSNR (dB)
borboleta	2x2	3.63	33.75
raposa	2x2	3.63	34.82
borboleta	3x3	5.89	30.43
raposa	3x3	5.89	31.23

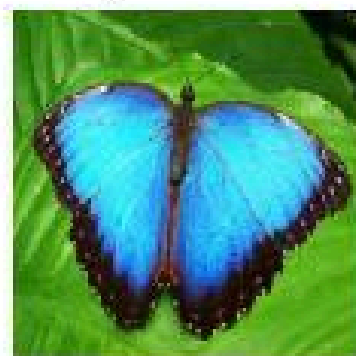
Original



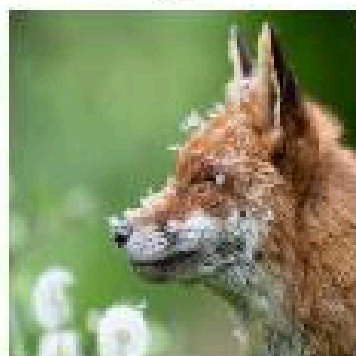
2x2 | Taxa = 3.63



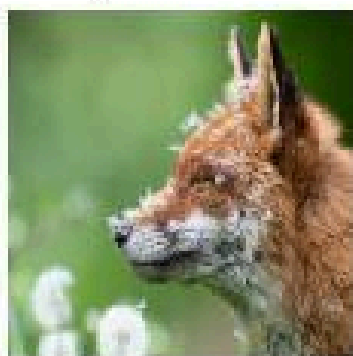
3x3 | Taxa = 5.89



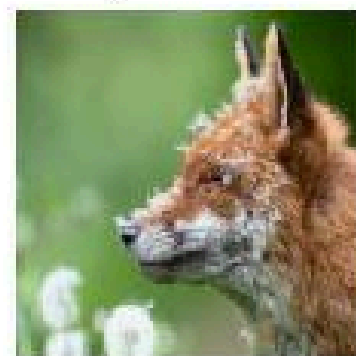
Original



2x2 | Taxa = 3.63



3x3 | Taxa = 5.89



2.4 Análise dos Resultados

Os resultados mostram claramente que o tamanho do bloco utilizado na VQ influencia diretamente a qualidade da reconstrução e a taxa de compressão. Com blocos de 2x2, a imagem é dividida em partes menores e mais detalhadas, o que permite uma representação mais precisa pelos vetores do codebook. Por isso, as reconstruções apresentam valores de PSNR mais altos, indicando melhor qualidade visual, embora a taxa de compressão seja menor.

Quando blocos de 3×3 são utilizados, ocorre o inverso: como a imagem passa a ter menos blocos, a quantidade total de índices diminui e a taxa de compressão aumenta. No entanto, blocos maiores costumam reunir regiões com mais variação interna, o que prejudica a fidelidade da reconstrução e reduz o PSNR. Assim, confirma-se o trade-off esperado: blocos menores preservam a qualidade, enquanto blocos maiores privilegiam a compressão.

Ao comparar imagens em tons de cinza com imagens coloridas, nota-se que as coloridas apresentam PSNR um pouco menor. Isso acontece porque cada canal (R, G e B) é quantizado separadamente, e as pequenas perdas de cada componente acabam se somando no resultado final. Mesmo assim, as taxas de compressão obtidas foram semelhantes às das imagens em cinza.

Em resumo, os experimentos mostram exatamente o comportamento previsto pela teoria da VQ: blocos menores resultam em maior qualidade reconstruída, enquanto blocos maiores aumentam a compressão, porém com maior perda visual.

3. Segmentação

Nesta etapa do trabalho, foram implementados dois métodos distintos de segmentação aplicados a duas imagens coloridas. O objetivo é comparar abordagens baseadas em diferentes princípios: uma fundamentada exclusivamente em agrupamento no espaço de cores, e outra que utiliza propriedades de textura, incorporando informações adicionais sobre padrões locais da imagem.

A segmentação é uma tarefa fundamental no processamento de imagens, pois permite dividir a imagem em regiões homogêneas que representam objetos, fundos ou áreas de interesse. Neste trabalho, optou-se por métodos que produzem resultados estruturados e que seguem diretamente o que foi solicitado pelo enunciado: um método baseado em agrupamento (k-means) e um método que utilize textura.

Além disso, em ambos os métodos foram testados **diferentes valores de K**, permitindo avaliar como a granularidade da segmentação influencia a qualidade e o nível de detalhamento obtido.

3.1 Metodologia

A metodologia foi organizada em duas abordagens independentes, cada uma avaliada com três valores distintos de K (2, 3 e 4).

a) Segmentação por K-means no Espaço de Cor Lab

A imagem é inicialmente convertida do espaço RGB para o espaço Lab, que separa a luminância dos componentes cromáticos e melhora a distinção entre cores semelhantes. Cada pixel é representado pelo vetor (L, a, b) e todos os pixels são reunidos em uma matriz de características.

Em seguida, aplicam-se sucessivamente diferentes valores de K no algoritmo K-means, produzindo segmentações com níveis variados de separação entre regiões. Cada resultado é

reorganizado no formato da imagem original, gerando mapas de clusters com diferentes granularidades.

Esse método utiliza apenas a cor como critério de separação, sendo eficaz para segmentar objetos visualmente distintos ou regiões com tonalidades marcadas.

b) Segmentação baseada em Textura com Filtros de Gabor

Para incorporar informações de textura, utiliza-se um conjunto de filtros de Gabor com múltiplas orientações e frequências espaciais. Essas respostas são capazes de detectar padrões repetitivos e estruturas locais como bordas, rugosidade e variações de orientação.

Após converter a imagem para tons de cinza, cada filtro é aplicado e gera um mapa de textura. Os valores de resposta são vetorizados, normalizados e utilizados como entrada para o algoritmo K-means. Assim como no método anterior, diferentes valores de K são testados (2, 3 e 4), resultando em segmentações mais grossas ou mais detalhadas.

Esse método é capaz de separar regiões que possuem texturas distintas mesmo quando têm cores semelhantes, sendo complementar à segmentação baseada somente em cor.

3.2 Código

```
1  function seg = segment_kmeans(img, K)
2
3      % converter para Lab
4      cform = makecform('srgb2lab');
5      lab = applycform(img, cform);
6
7      % reorganizar pixels
8      X = reshape(lab, [], 3);
9
10     % k-means
11     idx = kmeans(X, K, 'MaxIter', 200);
12
13     % voltar para imagem
14     seg = reshape(idx, size(img,1), size(img,2));
15
16 end
17
```

```

1 function seg = segment_gabor(img, K)
2
3     gray = im2double(rgb2gray(img));
4     [H, W] = size(gray);
5
6     % ----- PARÂMETROS DOS FILTROS GABOR -----
7     wavelengths = [3 5 9 15];
8     orientations = [0 pi/4 pi/2 3*pi/4];
9
10    numW = length(wavelengths);
11    numO = length(orientations);
12    numFilters = numW * numO;
13
14    responses = zeros(H, W, numFilters);
15    idx = 1;
16
17    % ----- GERAÇÃO E APLICAÇÃO DOS FILTROS -----
18    for w = 1:numW
19        lambda = wavelengths(w);
20
21        for o = 1:numO
22            theta = orientations(o);
23
24            % Gerar kernel Gabor manualmente
25            g = build_gabor_kernel(lambda, theta);
26
27            % Convolução
28            responses(:, :, idx) = abs(imfilter(gray, g, 'symmetric'));
29            idx = idx + 1;
30        end
31    end
32
33    % Vetorizar resposta
34    X = reshape(responses, [], numFilters);
35
36    % Normalizar
37    X = X - min(X(:));
38    X = X ./ max(X(:));
39
40    % K-means
41    labels = kmeans(X, K, 'MaxIter', 200);
42
43    % Voltar para imagem
44    seg = reshape(labels, H, W);
45
46 end

```

```

49 % -----
50 % Função auxiliar: cria um kernel Gabor de tamanho 31x31
51 % -----
52 function g = build_gabor_kernel(lambda, theta)
53
54     sigma = 0.56 * lambda;
55     gamma = 0.5;
56     psi = 0;
57
58     N = 31; % tamanho do kernel
59     [x, y] = meshgrid(-floor(N/2):floor(N/2));
60
61     % Rotação
62     x_theta = x * cos(theta) + y * sin(theta);
63     y_theta = -x * sin(theta) + y * cos(theta);
64
65     % Formula do Gabor
66     g = exp(-(x_theta.^2 + gamma^2 * y_theta.^2) / (2*sigma^2)) .* ...
67         cos(2*pi * x_theta / lambda + psi);
68
69 end

```

```

1 - clear all; close all; clc;
2
3 - imgs = {'borboleta.jpg', 'raposa.jpg'};
4
5 % Valores de K para testar
6 - K_values = [2 3 4];
7
8 - for i = 1:length(imgs)
9
10 -     img = im2double(imread(imgs{i}));
11
12 -     figure('Name', ['Segmentacao - ' imgs{i}], 'NumberTitle', 'off');
13 -     subplot(2, length(K_values)+1, 1);
14 -     imshow(img);
15 -     title('Original');
16
17     % ----- MÉTODO 1: K-MEANS LAB -----|
18 -     for k = 1:length(K_values)
19 -         K = K_values(k);
20 -         seg = segment_kmeans(img, K);
21
22 -         subplot(2, length(K_values)+1, 1+k);
23 -         imshow(seg, []);
24 -         title(['K-means K=' num2str(K)]);
25 -     end
26
27     % ----- MÉTODO 2: GABOR + K-MEANS -----
28 -     for k = 1:length(K_values)
29 -         K = K_values(k);
30 -         seg = segment_gabor(img, K);
31
32 -         subplot(2, length(K_values)+1, length(K_values)+1 + k);
33 -         imshow(seg, []);
34 -         title(['Gabor K=' num2str(K)]);
35 -     end
36
37 - end
38

```

3.3 Resultados Obtidos e Análise

Os resultados obtidos mostram diferenças claras entre os métodos de segmentação baseados em cor (K-means no espaço Lab) e aqueles baseados em textura (filtros de Gabor). Além disso, a variação do número de clusters K influencia diretamente o nível de detalhe das regiões segmentadas.

3.3.1 Imagens originais

Borboleta - Original



Raposa - Original



K-means (Lab) – Imagem da Borboleta

Para a borboleta, o método baseado apenas em cor teve desempenho satisfatório.

- **K = 2:** A segmentação separa de forma nítida o fundo e a borboleta, criando uma divisão simples, porém eficaz. A forma do inseto permanece bem definida.
- **K = 3:** Surge uma subdivisão interna da borboleta, separando áreas mais claras e mais escuras das asas. O fundo também é dividido em mais uma região.
- **K = 4:** A segmentação se torna mais rica em detalhes, incluindo nuances de tons tanto nas asas quanto no fundo. Apesar disso, alguns ruídos começam a aparecer, especialmente na região externa da imagem.

Em geral, o método K-means captou bem as diferenças de cor e luminosidade da borboleta, mesmo sem considerar textura.

3.3.2 Borboleta K-Means

K-means K = 2



K-means K = 3



K-means K = 4



Gabor (Textura) – Imagem da Borboleta

Nos resultados baseados em textura, o comportamento muda significativamente.

- **K = 2:** A segmentação destaca padrões de textura nas asas, mas também introduz ruídos decorrentes das respostas dos filtros.

- **K = 3:** Algumas estruturas internas da borboleta começam a ser separadas com maior consistência, mas o ruído ainda permanece evidente.
- **K = 4:** Surge mais definição na silhueta e nas áreas texturizadas, porém a segmentação ainda mantém regiões ruidosas, especialmente no fundo.

O método baseado em Gabor é muito sensível a variações de textura e, neste caso, acaba segmentando várias regiões que não correspondem a objetos reais, mas sim aos padrões do filtro. Ainda assim, ele resalta detalhes internos das asas que não aparecem no K-means de cor.

3.3.3 Borboleta Gabor + K-Means



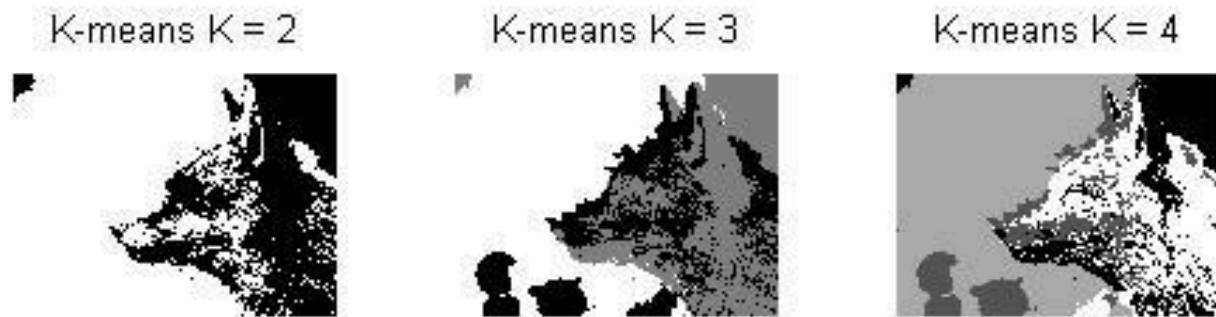
K-means (Lab) – Imagem da Raposa

Para a raposa, que tem variações relevantes de cor entre o animal e o fundo, o método funciona bem.

- **K = 2:** Já é possível separar o animal do fundo, embora algumas partes fiquem agrupadas de forma simplificada.
- **K = 3:** O contorno da raposa aparece de maneira mais fiel, com separação mais clara das sombras e detalhes do pescoço.
- **K = 4:** A segmentação fica mais detalhada, diferenciando ainda mais regiões do pelo e partes do fundo. Porém, assim como na borboleta, o aumento de K também traz pequenas regiões quebradas e ruídos.

O método de cor consegue destacar a raposa de forma eficiente, mostrando uma boa separação entre fundo, pelo claro e pelo escuro.

3.3.4 Raposa K-Means



Gabor (Textura) – Imagem da Raposa

Como na borboleta, o método baseado em textura produz resultados mais irregulares.

- **K = 2:** Os padrões de textura do filtro dominam a imagem, com grande presença de ruído. Ainda assim, algumas áreas da raposa se destacam do fundo.
- **K = 3:** A silhueta da raposa torna-se mais perceptível, e diferentes regiões de textura começam a ser agrupadas de maneira mais coerente.
- **K = 4:** Surgem regiões mais próximas do formato real do animal, com subdivisão de áreas texturizadas no pelo. No entanto, o ruído permanece significativo.

De forma geral, o método baseado em textura é mais adequado para imagens com padrões repetitivos bem definidos. A imagem da raposa, por ter texturas mais suaves, não se beneficia tanto dos filtros Gabor quanto a segmentação por cor.

3.3.5 Raposa Gabor + K-Means



3.4 Conclusão da Análise

Os experimentos mostram que:

- K-means no espaço Lab produz segmentações mais limpas, estruturadas e visualmente coerentes, destacando corretamente os objetos principais (borboleta e raposa).
- O método baseado em textura gera segmentações mais fragmentadas, capturando padrões de textura, mas também introduzindo ruídos consideráveis.
- A variação de K controla o nível de detalhe: valores baixos criam segmentações mais simples; valores maiores aumentam a granularidade, mas podem introduzir ruído.
- Para as imagens utilizadas, a segmentação por cor foi mais eficaz, enquanto o método baseado em textura foi mais útil apenas para ressaltar detalhes internos e padrões finos.